

“Design and Development of a Multi-Vendor Campus Marketplace & Delivery Platform”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

V. BHARDWAJ :2520090182

P. UDAY :2520090089

A. CHAITANYA :2520090094

Under the Esteemed Guidance of

Mrs. Divya Priya Degala

Assistant Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING
&
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled “**Design and Development of a Multi-Vendor Campus Marketplace & Delivery Platform**” is a record of Bonafide work of **V. BHARDWAJ - 2520090182, P. UDAY - 2520090089, A. CHAITANYA - 2520090094** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

V. BHARDWAJ – 2520090182

P. UDAY – 2520090089

A. CHAITANYA – 2520090094

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the project based report entitled “**Design and Development of a Multi-Vendor Campus Marketplace & Delivery Platform**” is a bonafide work done and submitted by **V. BHARDWAJ - 2520090182, P. UDAY - 2520090089, A. CHAITANYA - 2520090094** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2025-2026**.

Signature of the Guide

(Mrs. Divya Priya Degala)

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish is to express our sincere thanks to all those who have assisted us in one way or the other for the completion of our project.

Our greatest appreciation goes to our guide Mrs. Divya Priya Degala, Assistant Professor, Department of Computer Science and Engineering, which cannot be expressed in words, for her tremendous support, encouragement and guidance throughout this project.

We express our gratitude to the Head of the Department for Computer Science Engineering / Department for Computer Science and Information Technology for providing us with adequate facilities, ways and means by which we were able to complete this project-based laboratory work.

We thank all the members of the teaching and non-teaching staff, and also those who have assisted us directly or indirectly for the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

V. BHARDWAJ – 2520090182

P. UDAY – 2520090089

A. CHAITANYA – 2520090094

ABSTRACT

University campuses function as small, self-contained towns. Within a single campus there are canteens, stationery shops, printing and xerox centres, laundry services, fruit and snack stalls, and a large population of students who buy and sell used books, calculators, cycles, lab coats and electronics among themselves. Yet almost all of this commerce is still coordinated through scattered WhatsApp groups, notice boards, word of mouth and cash payments. Orders are missed, prices are inconsistent, sellers cannot track demand, and students waste time standing in queues between classes. This project addresses that gap through the Design and Development of a Multi-Vendor Campus Marketplace & Delivery Platform.

The proposed system is a single web-based platform on which multiple independent vendors — campus shops, canteens, service providers and verified student sellers — can register, list their products or services, manage their own inventory, and receive orders, while buyers browse a unified catalogue, compare prices across vendors, place orders, pay online or on delivery, and track the order until it reaches their hostel block, department or classroom. The platform is built around four principal roles: the Customer, the Vendor, the Delivery Partner (a student or staff member who fulfills deliveries in free slots) and the Administrator, each with a dedicated dashboard and role-based permissions.

Functionally the system is organised into interconnected modules: authentication and verified campus-email onboarding, vendor and product management, a searchable multi-vendor catalogue with filters and sorting, a cart capable of splitting a single checkout into vendor-wise sub-orders, an order lifecycle engine, a delivery assignment and live status module, a secure payment module, and a ratings and review module that keeps vendor quality transparent. An analytics dashboard gives vendors and administrators visibility into sales, peak hours, popular items and delivery performance.

The system is implemented using a modern, responsive technology stack — React.js on the front end, Node.js with Express on the back end, MongoDB for persistence, JSON Web Tokens for stateless authentication, and a payment gateway integration for online transactions — and is deployed on cloud infrastructure so that it is accessible from any laptop or smartphone on campus. The result is a complete, reliable and user-friendly digital solution that reduces waiting time, formalises campus commerce, creates small earning opportunities for student delivery partners, and gives every campus vendor an equal digital shopfront.

INDEX

S. No.	Chapters	Topics	Page No.
	Acknowledgement		
	Abstract		
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objective(s) 1.5 Importance of the application 1.6 Target users / audience	1-12
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools and frameworks	13-17
3	Technology Stack	3.1 Front-end 3.2 Back-end 3.3 Database 3.4 Version control 3.5 APIs / External services	18-24
4	System Architecture	4.1 High-level architecture 4.2 Description of each layer 4.3 Deployment architecture	25-29
5	Design	5.1 ER Diagram / Database schema	30-33
6	Implementation	6.1 Module-wise implementation 6.2 Front-end logic 6.3 API integration 6.4 Authentication & Authorization	34-41
7	Features	7.1 List of main features 7.2 How each feature works	42-48
8	Testing	8.1 Postman testing 8.2 Integration testing 8.3 Tools used 8.4 Test cases and results	49-55
9	Deployment	9.1 Steps to deploy 9.2 Environment configuration 9.3 Hosting of frontend and backend	56-60
10	Challenges & Limitations	10.1 Issues faced 10.2 Solutions applied 10.3 Known limitations	61-65
11	Future Enhancements	11.1 Planned features 11.2 Possible integrations	66-69
12	Conclusion	12.1 Summary 12.2 What was achieved 12.3 Skills learned	70-73
13	References	Books, tutorials, APIs, documentation sites used	74
14	Appendices	Screenshots, code snippets, setup instructions, user manual, review forms	75-80

CHAPTER – 1 INTRODUCTION

1.1 Background of the Project

A modern university campus is not merely an academic space; it is a dense micro-economy. Thousands of students and staff live, eat, study and shop within a boundary of a few square kilometres. Canteens and food courts serve peak rushes between lectures, stationery and print shops handle record books and assignment submissions, laundry and salon services operate on subscription-like patterns, and an informal resale economy moves textbooks, drafters, lab coats, calculators, cycles and electronics from senior students to juniors every semester.

Despite the density of this activity, the coordination layer behind it has remained almost entirely manual. Students discover what is available by walking to a shop or by scrolling through unstructured WhatsApp groups. Vendors accept orders verbally, note them on paper, and have no record of demand patterns. Payments are largely in cash, which produces disputes over change and no receipt trail. Second-hand sales depend on being in the right group at the right time, so many usable items are simply thrown away at the end of the year.

Meanwhile, the commercial world outside the campus gate has moved decisively to aggregated, multi-vendor platforms. Food delivery, grocery, quick-commerce and resale marketplaces have normalised the expectation that a catalogue, a price comparison, an online payment and a live tracking screen should exist for any purchase. Students carry this expectation onto campus, where it is not met. External platforms cannot fill the gap either: their delivery partners frequently cannot enter hostel blocks or academic buildings, their minimum order values and delivery fees are unsuitable for a fifty-rupee purchase, and they do not list the campus stationery shop or the senior selling a used drafter at all.

The idea for this project emerged from these observations. The advances that made external marketplaces possible — component-based front-end frameworks, lightweight REST back ends, document databases, token-based authentication, hosted payment gateways and free-tier cloud deployment — are now accessible to a student team. What is required is not new technology but a correctly modelled domain: a platform that understands a campus rather than a city, that treats hostel blocks and department buildings as delivery addresses, that verifies identity through institutional email, and that allows many small vendors to coexist under one catalogue.

From an academic and engineering perspective, the background of this project lies at the intersection of several core areas of computer science, each of which is exercised in the course of building the system:

1. **Web Technology:** a component-driven, responsive front end built with React.js, HTML5, CSS3 and JavaScript, communicating with a dynamic server-side application.
2. **Database Management:** normalised modelling of users, vendors, products, carts, orders and deliveries, with indexing for search and aggregation for analytics.
3. **API Design and Integration:** a documented RESTful interface, plus integration with third-party payment, notification and mapping services.
4. **Information Security:** password hashing, JWT-based stateless sessions, role-based access control and protection of payment data.

5. Software Engineering Principles: modular architecture, version control discipline, testing and maintainability.
6. User Experience Design: task-oriented navigation designed for a user who is standing between two lectures with one hand on a phone.

In a broader context, the project is aligned with the Digital India initiative and with the wider institutional shift towards cashless, auditable and data-driven campus operations. It also carries a social dimension: by formalising delivery work into short, schedulable slots, it creates a legitimate small earning opportunity for students, and by giving every small campus vendor an equal digital shopfront, it prevents the visibility monopoly that advertising-driven platforms create.

1.2 Problem Statement

Although students and vendors on a campus transact with each other many hundreds of times a day, there is no common digital system that connects them. The resulting problems are structural rather than incidental, and they affect buyers, sellers and the institution simultaneously.

For the buyer, the first problem is discovery. There is no single place to find out which canteen still has items in stock, what the stationery shop charges for spiral binding, or whether any senior is selling a second-hand drafter. Information is scattered across notice boards, group chats and personal networks, so purchase decisions are made with incomplete knowledge and without any price comparison. The second problem is time. Peak demand on a campus is extremely concentrated — a lunch break may be forty minutes long, during which the entire student body attempts to buy food from a small number of counters. Queueing consumes the break itself.

For the vendor, the problems are visibility and record-keeping. A shop located near the main gate is seen by everyone; a shop behind the workshop block is seen by almost nobody, irrespective of quality or price. Orders taken verbally are forgotten or duplicated. Because no record exists, the vendor cannot tell which items sold out early, which stayed unsold, or which hours produce the most revenue, and therefore cannot plan stock. Cash handling introduces further friction and a persistent shortage of small change.

For the institution, the absence of a system means an absence of accountability. There is no auditable trail of transactions, no structured mechanism for complaints about quality or overcharging, and no data on which to base decisions about vendor contracts, operating hours or additional outlets.

A further problem is trust in peer-to-peer resale. Informal second-hand sales depend on personal acquaintance; a buyer has no way to verify that a seller is a genuine student of the institution, and neither party has recourse if an item is misrepresented. Consequently, a large volume of reusable material is discarded each year rather than resold.

Finally, there is a technical integration problem. Even where individual vendors have attempted digital solutions — a phone number for orders, a Google Form, a UPI QR code — these solutions are isolated. A student ordering lunch from one vendor and a printout from another must conduct two entirely separate transactions, with no unified cart, no common payment record and no single delivery.

The project therefore aims to achieve the following problem resolutions:

1. Eliminate fragmentation by bringing all campus vendors, products and services into one searchable catalogue.
2. Reduce waiting time through pre-ordering, scheduled pickup and in-campus delivery to hostel blocks and departments.
3. Give every vendor equal digital visibility along with sales and inventory records.
4. Establish trust in peer-to-peer resale through institutional-email verification and a review system.
5. Provide secure, cashless payment with a verifiable transaction record for buyers, vendors and administration.
6. Enable data-driven campus decisions through analytics on demand, peak hours and delivery performance.

1.3 Scope of the Project

The scope of this project extends across the technological, functional, economic and social dimensions of campus commerce. The central aim is to establish a comprehensive, web-based platform that consolidates the buying, selling and delivery activity of a university campus into a single interface serving four distinct classes of user.

The technical scope involves building a fully responsive, database-driven web application that functions efficiently on laptops, tablets and smartphones. The front end is developed using React.js with HTML5, CSS3 and JavaScript; the back end is implemented as a RESTful API using Node.js and Express.js; MongoDB serves as the primary data store; and authentication is handled through JSON Web Tokens with hashed credentials. The application integrates a payment gateway in test mode, a transactional email service for notifications, and cloud storage for product images.

The functional scope covers the complete order lifecycle. A customer registers with an institutional email, browses a unified catalogue, filters by vendor, category, price and availability, adds items from multiple vendors to a single cart, and checks out. The system automatically splits the checkout into vendor-wise sub-orders so that each vendor sees only its own items. Vendors accept, prepare and mark orders ready. A delivery partner picks up and delivers to a campus location, updating status at each step. Customers rate the vendor and the delivery. Administrators approve vendors, resolve disputes, and view platform-wide analytics.

The scope also includes a peer-to-peer resale sub-module, in which a verified student may list a used item for sale, subject to administrator approval, using the same catalogue, chat and payment infrastructure as commercial vendors.

Certain boundaries are deliberately placed on the scope. The platform operates within a single campus and does not attempt to serve external addresses. Deliveries are performed by registered student or staff partners on foot or by cycle, not by a commercial fleet. Payment gateway integration is implemented and demonstrated in sandbox mode, since live settlement requires a registered merchant account held by the institution. Native Android and iOS applications are identified as future work; the current deliverable is a responsive progressive web application.

1.4 Objectives

The principal objective of the project is to design, develop and deploy a functional multi-vendor marketplace and delivery platform tailored to a university campus. This broad aim is decomposed into the following specific objectives:

1. To design a secure registration and authentication mechanism that verifies users through institutional email identity and distinguishes between customer, vendor, delivery partner and administrator roles.
2. To develop a vendor onboarding and storefront module that allows an approved vendor to manage a catalogue, set prices, control stock and define operating hours independently.
3. To build a unified, searchable product catalogue that aggregates listings from all vendors with filtering, sorting and price comparison across vendors.
4. To implement a cart and checkout engine capable of splitting a single customer transaction into independent vendor-wise sub-orders while presenting the customer with one payment.
5. To design an order lifecycle state machine covering placement, acceptance, preparation, pickup, delivery and completion, with validated transitions and timestamps.
6. To implement a delivery assignment module that allows student delivery partners to view, accept and complete delivery tasks, with live status visible to the customer.
7. To integrate a secure online payment gateway alongside a cash-on-delivery option, and to maintain a verifiable transaction record for every order.
8. To provide a ratings and reviews mechanism that makes vendor and delivery quality transparent and accountable.
9. To build analytics dashboards that present sales, order volume, peak-hour demand and delivery performance to vendors and administrators.
10. To ensure responsiveness, accessibility and acceptable performance on the low-bandwidth mobile connections typical of a crowded campus network.
11. To validate the system through unit, integration and user-acceptance testing, and to deploy it on cloud infrastructure with continuous integration.

1.5 Importance of the Application

The importance of this application can be examined from the perspective of each stakeholder it serves. For students, the most immediate benefit is the recovery of time. A twenty-minute queue during a forty-minute break is a significant loss; pre-ordering and delivery convert that period back into usable time. Price comparison across vendors, which is currently impossible without physically visiting each shop, allows students on constrained budgets to make informed choices. The resale sub-module gives juniors access to affordable used equipment and gives seniors a way to recover value from items they would otherwise discard.

For vendors, the platform functions as a business tool rather than merely a sales channel. Equal digital placement removes the disadvantage of poor physical location. Structured order records eliminate the duplication and forgetting inherent in verbal ordering. Sales analytics allow demand-based stocking, which reduces both wastage of perishable items and lost sales through stock-outs. Cashless settlement removes the change problem and provides accounts that a small vendor can actually use.

For delivery partners, the platform creates a flexible, legitimate earning opportunity structured around class schedules. A student with a free hour can accept a small number of delivery tasks without any fixed commitment, which is a form of employment not otherwise available inside a campus.

For the institution, the platform provides oversight without micromanagement. Administrators can verify which vendors are operating, track complaint patterns, monitor pricing compliance, and base decisions about contracts and outlet placement on actual demand data. The auditable transaction trail also supports transparency requirements.

There is, additionally, an environmental and social dimension. Consolidated in-campus delivery on foot or by cycle replaces a large number of individual trips and external delivery vehicles entering campus. The resale module directly extends the life of books, instruments and electronics. And by giving the smallest campus vendor the same catalogue placement as the largest, the platform supports an equitable rather than advertising-driven marketplace.

1.6 Target Users / Audience

The system is designed for four primary categories of user, each with distinct needs, permissions and interface requirements.

- Customers (students, faculty and staff): the largest user group. They browse, compare, order, pay, track and review. Their primary constraints are limited time, limited budget and mobile-first access on a congested network.
- Vendors (campus shops, canteens, service providers and verified student sellers): they manage catalogues, stock, prices and operating hours, accept and fulfil orders, and analyse their own sales. Many have limited technical background, so their dashboard is designed for simplicity.
- Delivery Partners (students or staff working in free slots): they view available delivery tasks, accept them, collect from vendors and deliver to campus locations, updating status at each stage.
- Administrators (institutional staff managing the platform): they approve vendor and resale listings, manage users, resolve disputes, monitor platform health and access aggregate analytics.

Secondary audiences include the institutional administration, which benefits from transactional oversight and demand data, and the academic evaluators of this project, for whom the system demonstrates applied competence in full-stack web development, database design, API integration, security and deployment.

CHAPTER – 2 SYSTEM REQUIREMENTS

System requirements define the minimum hardware, software and tooling environment necessary to develop, deploy and operate the platform. They are divided into requirements for the development machine, the deployment server and the end-user device, since these three environments have materially different needs.

2.1 Hardware Requirements

Because the application is a web-based client-server system, the computational load is distributed. The end-user device performs only rendering; the server performs application logic and database operations.

Component	Minimum (Development)	Recommended (Development)
Processor	Intel Core i3 (8th gen) / AMD Ryzen 3	Intel Core i5 / i7 or AMD Ryzen 5 and above
RAM	8 GB	16 GB (Node, MongoDB, VS Code and a browser run concurrently)
Storage	128 GB SSD with 20 GB free	256 GB SSD or higher
Display	1366 × 768	1920 × 1080 for responsive testing
Network	5 Mbps broadband	20 Mbps or higher

Environment	Requirement
Deployment server	2 vCPU, 2–4 GB RAM, 20 GB SSD (cloud instance or managed platform free/starter tier)
Database	Managed MongoDB cluster (Atlas M0 for development, M10 or higher for production load)
End-user device	Any smartphone, tablet or laptop with 2 GB RAM and a modern browser; no installation required

2.2 Software Requirements

Category	Software	Version / Notes
Operating System	Windows 10/11, Ubuntu 20.04+ or macOS 12+	Development is platform-independent
Runtime	Node.js	v18 LTS or higher, with npm 9+
Database	MongoDB	v6.0+ locally, or MongoDB Atlas cloud cluster
Browser	Chrome, Edge or Firefox	Latest stable; used for testing and DevTools
API Client	Postman	For endpoint testing and collection-based regression runs
Editor	Visual Studio Code	With ESLint, Prettier and MongoDB extensions
Version Control	Git and GitHub	Branch-per-feature workflow
Containerisation	Docker (optional)	For reproducible local environments

2.3 Development Tools and Frameworks

- React.js with Vite — component-based front-end framework and fast development server with hot module replacement.
- React Router — declarative client-side routing for role-specific dashboards and protected routes.
- Tailwind CSS — utility-first styling for a consistent, responsive design system without large custom stylesheets.

- Axios — promise-based HTTP client with request/response interceptors used to attach authentication tokens.
- Node.js and Express.js — server runtime and minimal web framework hosting the REST API.
- Mongoose — object document mapper providing schema validation, population and middleware over MongoDB.
- JSON Web Token and bcrypt — stateless session tokens and salted password hashing.
- Multer and Cloudinary — multipart upload handling and cloud image storage for product photographs.
- Socket.IO — real-time channel used for live order status and delivery tracking updates.
- Nodemailer — transactional email for verification links and order confirmations.
- Jest and Supertest — unit and HTTP integration testing of the API layer.
- Postman — manual and automated endpoint testing with environment variables per deployment.
- Git, GitHub and GitHub Actions — version control, code review and continuous integration.
- Figma and draw.io — interface prototyping and architecture/ER diagramming.

CHAPTER – 3 TECHNOLOGY STACK

The technology stack was selected on four criteria: suitability to a real-time, data-driven marketplace; availability of mature documentation and community support; a free or low-cost path to deployment appropriate to a student project; and relevance to current industry practice so that the skills acquired transfer beyond the academic context. The resulting stack is the MERN stack extended with real-time and third-party services.

3.1 Front-End

The client application is built with React.js. React's component model maps naturally onto the interface of a marketplace, where the same building blocks — a product card, a vendor header, an order status chip, a rating widget — recur throughout the application in different combinations. Building these as isolated, reusable components reduced duplication substantially and made the interface consistent across the four role-specific dashboards.

Application state is managed through React's own hooks for local concerns and through the Context API for cross-cutting state such as the authenticated user, the active cart and the theme. A dedicated cart context holds items keyed by vendor, which allows the checkout view to render vendor-wise sub-totals without recomputation. Routing is handled by React Router, with a higher-order protected-route component that inspects the decoded token and redirects unauthorised users before a dashboard is ever mounted.

Styling uses Tailwind CSS, which enforces a single spacing, colour and typography scale across the application and keeps responsive behaviour local to the markup through breakpoint prefixes. The interface is designed mobile-first, since the overwhelming majority of campus traffic originates from phones. Lazy loading of route bundles and of product images keeps the first meaningful paint fast on congested campus Wi-Fi.

3.2 Back-End

The server is implemented in Node.js with the Express.js framework. Node's non-blocking, event-driven model is well matched to the workload of this application, which is dominated by concurrent I/O — database queries, payment gateway calls and notification dispatch — rather than by computation. Using JavaScript on both sides of the application also removed context switching between languages and allowed validation logic and type definitions to be shared.

The API follows REST conventions, exposing resource-oriented routes such as `/api/vendors`, `/api/products`, `/api/orders` and `/api/deliveries`, with HTTP verbs carrying the semantics of the operation and standard status codes reporting the outcome. The server is organised in layers: routes declare endpoints, controllers handle request and response shaping, services contain business logic such as order splitting and status transition validation, and models define the data schema. Middleware handles authentication, role authorisation, request validation, rate limiting and centralised error handling.

Real-time behaviour is delivered through Socket.IO. When a vendor accepts an order or a delivery partner updates a status, the server emits an event to the room corresponding to that order, and the customer's tracking screen updates without polling.

3.3 Database

MongoDB was selected as the primary data store. A marketplace catalogue is inherently heterogeneous: a plate of food has a preparation time and a vegetarian flag, a printout has page count and binding options, and a second-hand calculator has a condition grade and a purchase year. A document model accommodates this variation naturally within one products collection, whereas a rigid relational schema would require either many sparse columns or a proliferation of specialised tables.

Mongoose provides schema definitions, validation, middleware hooks and population over MongoDB. Relationships between users, vendors, products and orders are modelled by reference, while data that is always read together — order line items, delivery status history, the delivery address snapshot — is embedded within the order document so that an entire order can be retrieved in a single read. Compound indexes on vendor and category, a text index on product name and description, and an index on order status and creation time support the platform's most frequent queries. Aggregation pipelines generate the analytics dashboards directly in the database.

3.4 Version Control

The project is version-controlled with Git and hosted on GitHub. The team follows a branch-per-feature workflow: the main branch always holds deployable code, a develop branch integrates completed work, and each module is built on a feature branch that is merged through a pull request after review by at least one other team member. Commit messages follow the conventional-commits style so that the history is readable and a changelog can be generated. Environment files and dependency directories are excluded through .gitignore, and secrets are never committed. GitHub Issues track tasks and defects, and a project board maps them to development phases.

3.5 APIs and External Services

- Payment gateway (Razorpay, sandbox mode): order creation, hosted checkout and server-side signature verification for online payments, alongside a cash-on-delivery path.
- Cloudinary: cloud storage, transformation and CDN delivery of product images, keeping large binaries out of the database.
- Nodemailer with an SMTP provider: email verification of institutional addresses, order confirmations and password reset links.
- Socket.IO: bidirectional real-time channel for order status and delivery tracking.
- Leaflet with OpenStreetMap: rendering of a campus map with annotated block-level delivery points.
- MongoDB Atlas: managed database hosting with automated backups and monitoring.
- Render / Vercel: managed hosting for the API server and the static front-end build respectively.

CHAPTER – 4 SYSTEM ARCHITECTURE

4.1 High-Level Architecture

The platform follows a three-tier client-server architecture with a clear separation between presentation, application logic and persistence. The tiers communicate only through defined interfaces: the client speaks to the server exclusively over HTTPS REST calls and a WebSocket channel, and the server is the only component with database credentials. This separation allows any tier to be scaled, replaced or tested independently.

The logical flow of the system is as follows:

- **Presentation Tier** — the React single-page application served as a static bundle from a CDN, rendering four role-specific dashboards from a shared component library.
- **Application Tier** — the Node.js/Express REST API, containing authentication middleware, role guards, business services (catalogue, cart, order splitting, delivery assignment, payment verification, analytics) and the Socket.IO event server.
- **Data Tier** — the MongoDB cluster holding users, vendors, products, carts, orders, deliveries, payments and reviews, supported by Cloudinary for binary assets.
- **External Services** — payment gateway, email service and map tiles, all invoked from the application tier and never directly from the browser with secret credentials.

A representative request traverses the architecture in the following sequence: the browser issues an authenticated request carrying a JWT in the Authorization header; an Express middleware verifies the signature and attaches the decoded user to the request; a role guard confirms that the user's role permits the operation; a validation middleware checks the payload shape; the controller delegates to a service; the service reads or writes through Mongoose models; and the response is serialised back to the client while any resulting state change is broadcast over the WebSocket channel to interested parties.

4.2 Description of Each Layer

Front-end layer. The React application is composed of pages, feature components and shared UI primitives. Pages correspond to routes; feature components encapsulate a domain concern such as the cart drawer or the order timeline; primitives are stateless and styled through Tailwind. A central Axios instance attaches the token to every outgoing request and intercepts 401 responses to trigger a silent refresh or a redirect to login. Loading, empty and error states are handled explicitly in every data-fetching component so that the interface degrades gracefully on a poor connection.

Back-end layer. The API server is organised into routes, controllers, services, models and middleware. Controllers remain thin, restricted to extracting parameters and shaping responses, while all business rules reside in services. The order service, for example, owns the logic that splits a mixed cart into vendor-wise sub-orders, recomputes totals, validates stock atomically and enforces the permitted transitions of the order state machine. Centralised error handling converts thrown application errors into consistent JSON responses with appropriate status codes, so the client never has to parse heterogeneous error shapes.

Database layer. MongoDB stores nine principal collections. Cross-collection consistency for the critical path — decrementing stock while creating an order — is protected with a transaction over a replica set,

so that a partially applied order cannot exist. Indexes are defined explicitly in the schemas rather than created ad hoc, ensuring that every deployment has identical query performance characteristics.

Security layer. Security is applied at each tier rather than at a single boundary: HTTPS in transit, bcrypt hashing at rest for credentials, signed and short-lived JWTs for sessions, role-based authorisation on every mutating endpoint, server-side re-validation of all prices and totals regardless of what the client submits, input sanitisation against injection, rate limiting on authentication routes, and signature verification of payment gateway callbacks.

4.3 Deployment Architecture

The production deployment uses managed cloud services rather than self-administered servers, which suits both the budget and the maintenance capacity of a student team. The front-end build is deployed to a static hosting platform with an integrated global CDN, providing automatic TLS and atomic rollbacks. The API server runs as a containerised web service on a managed platform that handles process supervision, health checks and horizontal scaling. The database runs on MongoDB Atlas with network access restricted to the API service and automated daily backups. Images reside in Cloudinary and are served from its CDN.

Continuous integration is configured through GitHub Actions. Every push to a feature branch triggers linting and the Jest test suite; a merge into main additionally triggers a production build and deployment of both the client and the server. Environment-specific configuration is injected through platform environment variables, so the same image runs in staging and production with different credentials and no code change. Application logs and uptime are monitored through the hosting platform's dashboards, with alerts on elevated error rates.

CHAPTER – 5 DESIGN

5.1 ER Diagram and Database Schema

The data model was derived by identifying the entities that exist in the campus commerce domain and the relationships between them. Nine collections form the core of the schema. The relationships are as follows: a User may own one Vendor profile; a Vendor has many Products; a User has one active Cart containing many Cart Items; a Cart produces one or more Orders, one per vendor; an Order has many Order Items and exactly one Payment; an Order has one Delivery assigned to one delivery partner; and a completed Order permits one Review of the vendor and one of the delivery partner.

Collection	Key Fields	Purpose
users	_id, name, email (institutional, unique), passwordHash, role, phone, hostelBlock, isVerified, createdAt	Identity and role assignment for all four user types
vendors	_id, ownerId → users, shopName, category, description, location, openingHours, isApproved, rating, commissionRate	Storefront profile of a shop, canteen, service or student seller
products	_id, vendorId → vendors, name, description, category, price, discountPrice, stock, unit, images[], isAvailable, attributes{}, textIndex	Catalogue listing; flexible attributes accommodate food, print, service and resale items
carts	_id, userId → users, items[{productId, vendorId, qty, priceSnapshot}], updatedAt	Single active cart per user, spanning multiple vendors
orders	_id, orderNumber, customerId, vendorId, items[], itemsTotal, deliveryFee, grandTotal, status, deliveryAddress{block, room, landmark}, statusHistory[], placedAt	One document per vendor sub-order, with an embedded audit trail
payments	_id, orderIds[], userId, method, gatewayOrderId, gatewayPaymentId, signature, amount, status, paidAt	One payment record may settle several vendor sub-orders from one checkout
deliveries	_id, orderId → orders, partnerId → users, pickupVendorId, dropLocation, status, acceptedAt, pickedUpAt, deliveredAt, otp	Assignment and lifecycle of a single delivery task
reviews	_id, orderId, authorId, targetType (vendor partner product), targetId, rating, comment, createdAt	Post-completion feedback, one per target per order
notifications	_id, userId, type, title, body, isRead, createdAt	In-app notification feed mirrored by email for critical events

Design decisions of note. First, an order is stored per vendor rather than per checkout, because the lifecycle of one vendor's items is entirely independent of another's — the canteen may still be preparing food when the stationery shop has already handed over a printout. A shared payment document links the sub-orders back to the single transaction the customer actually made. Second, prices are snapshotted into cart items and order items at the moment of addition, so that a later price change by a vendor cannot alter a placed order. Third, status history is embedded as an append-only array of transitions with timestamps and actor identifiers, giving a complete audit trail without a separate join. Fourth, a delivery OTP shared with the customer and verified by the partner at handover prevents false completion claims.

Normalisation is applied selectively. Referenced relationships are used where an entity is queried independently (users, vendors, products), while embedding is used where data is only ever read in the

context of its parent (order items, status history, address snapshot). This hybrid approach keeps the common read paths to a single query while avoiding duplication of mutable master data.

CHAPTER – 6 IMPLEMENTATION

6.1 Module-Wise Implementation

Development proceeded module by module, each module being built on a feature branch, tested in isolation and then integrated. The modules and their implementation are described below.

Authentication and onboarding module. Registration accepts only institutional email domains, which are validated against a configurable allow-list. Passwords are hashed with bcrypt using ten salt rounds before storage. A verification token is emailed and must be redeemed before login is permitted, which establishes that every account belongs to a genuine member of the institution. Role is assigned at registration; vendor and delivery-partner registrations additionally enter a pending state until an administrator approves them.

Vendor and product management module. An approved vendor receives a dashboard from which the storefront profile, opening hours and catalogue are managed. Product creation supports images uploaded through Multer and stored in Cloudinary, category assignment, price and discount price, stock quantity and a flexible attributes object for domain-specific fields. Stock is decremented transactionally at order placement and restored automatically if an order is cancelled before acceptance. A vendor may toggle availability of an individual product or of the entire storefront when closed.

Catalogue and search module. The public catalogue aggregates products across all approved vendors. Search is served by a MongoDB text index over name and description, combined with filters on category, vendor, price range and availability, and sorting by price, rating or preparation time. Results are paginated on the server to keep payloads small on mobile connections.

Cart and checkout module. The cart holds items from any number of vendors simultaneously, grouped visually by vendor with per-vendor subtotals. At checkout, the order service revalidates every price and stock level from the database, ignoring any values submitted by the client, then creates one order document per vendor within a single database transaction and links them all to one payment record.

Order lifecycle module. Order status is governed by an explicit state machine — PLACED, ACCEPTED, PREPARING, READY, OUT_FOR_DELIVERY, DELIVERED, with CANCELLED and REJECTED as terminal branches. Each transition is validated against a permitted-transitions map and against the role of the actor attempting it, so that, for example, a customer cannot mark an order delivered and a vendor cannot cancel an order that has already left the shop.

Delivery module. When an order reaches READY, a delivery task is published to the pool visible to on-duty delivery partners, showing pickup vendor, drop block and expected payout. The first partner to accept locks the task. Status updates emitted by the partner propagate to the customer in real time over Socket.IO. Completion requires the customer's OTP, which closes the order and credits the partner's earnings ledger.

Payment module. Online payment uses the gateway's server-created order identifier and hosted checkout. On completion the client returns a payment identifier and signature, which the server verifies cryptographically against its own secret before marking the payment successful. Cash on delivery is recorded as a pending payment settled by the delivery partner at handover.

Reviews and analytics module. After delivery, a customer may rate the vendor, the delivery partner and individual products. Aggregate ratings are recomputed with an aggregation pipeline. The analytics dashboards use pipelines that group orders by day, hour, product and status to produce revenue trends, peak-hour distribution, best-selling items and delivery time statistics.

6.2 Front-End Logic

The interface is composed as a hierarchy of components with data fetched at the page level and passed downward, while genuinely cross-cutting state is held in context providers. The authentication context holds the decoded user and exposes login, logout and refresh operations; the cart context exposes add, update, remove and clear operations and persists the cart to the server on every mutation so that it survives a device change.

Rendering is optimised for perceived speed. Each data-driven view renders a skeleton placeholder during loading rather than a blank screen, list components are memoised to avoid re-rendering the full catalogue when a single quantity changes, and route bundles are code-split so that a customer never downloads the vendor or administrator dashboards. Images are lazy-loaded and served in a compressed, width-appropriate format by the CDN.

Form handling performs validation on the client for immediate feedback while treating the server as the authority; every rule enforced in the browser is enforced again in the API. Real-time updates arrive through a Socket.IO client that subscribes to the rooms relevant to the current user and patches local state in place, so the order tracking screen updates without a refresh and without polling.

6.3 API Integration

All communication with the back end passes through a single configured Axios instance. A request interceptor attaches the access token; a response interceptor inspects failures, attempts a silent token refresh on a 401, retries the original request once, and otherwise clears the session and redirects to login. Errors are surfaced to the user through a shared toast mechanism with messages derived from the API's consistent error envelope.

Integration with third-party services is deliberately confined to the server wherever a secret is involved. The browser never holds the payment gateway secret, the Cloudinary API secret or the SMTP credentials; it receives only short-lived, scoped artefacts such as a gateway order identifier or a signed upload token. Payment verification, in particular, is performed entirely server-side, because a client-reported success is not evidence of payment.

The API surface is documented with request and response examples for every endpoint, which allowed front-end and back-end work to proceed in parallel against an agreed contract, with mock responses standing in until the corresponding endpoint was complete.

6.4 Authentication and Authorization

Authentication is stateless and token-based. On successful login the server issues a short-lived access token carrying the user identifier and role, together with a longer-lived refresh token stored in an HTTP-only cookie. The access token is verified on every protected request by a middleware that rejects expired or tampered tokens before any controller executes.

Authorization is enforced at two levels. Role-level guards restrict entire routes — only a vendor may create a product, only an administrator may approve a vendor, only a delivery partner may accept a delivery task. Resource-level checks then confirm ownership: a vendor may modify only products belonging to that vendor, and a customer may view only that customer's own orders. Both checks occur on the server; the corresponding hiding of controls in the interface is a usability measure, never a security measure.

Supporting protections include rate limiting on login and registration to frustrate brute-force attempts, account lockout after repeated failures, password reset through single-use expiring tokens, input sanitisation against NoSQL injection, security headers applied through Helmet, and a CORS policy that permits only the deployed front-end origin.

CHAPTER – 7 FEATURES

7.1 List of Main Features

- Institutional email verification and role-based registration for customers, vendors, delivery partners and administrators.
- Vendor onboarding with administrator approval and an independent, self-managed storefront.
- Unified multi-vendor catalogue with text search, category and price filters, and sorting.
- Cross-vendor price comparison for equivalent items.
- Multi-vendor cart with automatic splitting of checkout into vendor-wise sub-orders.
- Secure online payment through a gateway, plus a cash-on-delivery option.
- Real-time order tracking through the full status lifecycle.
- In-campus delivery to hostel blocks and departments by student delivery partners, with OTP-verified handover.
- Peer-to-peer resale of used books, instruments and electronics by verified students.
- Ratings and reviews for vendors, delivery partners and products.
- Vendor analytics: revenue trends, peak hours, best sellers and stock alerts.
- Administrator console: approvals, user management, dispute resolution and platform analytics.
- In-app and email notifications at each significant order event.
- Fully responsive, mobile-first interface with a dark mode.

7.2 How Each Feature Works

Registration and verification. From the user's perspective, the student enters name, institutional email, password and hostel block, then clicks a link in a verification email and logs in. Technically, the server validates the email domain against an allow-list, hashes the password with bcrypt, stores a hashed verification token with an expiry, dispatches the email through Nodemailer, and flips the verification flag only when the token is redeemed within its validity window.

Browsing and search. The user types a query and applies filters; results appear ranked and paginated. Technically, the query is executed against a MongoDB text index with filter predicates applied in the same query and a sort stage appended, returning a bounded page together with a total count used to render pagination.

Multi-vendor cart and checkout. The user adds a plate of rice from the canteen and a set of printouts from the print shop and sees a single cart with two vendor groups and one payable total. Technically, the cart stores items with a vendor reference and a price snapshot; at checkout the order service re-reads every product, revalidates price and stock, opens a database transaction, creates one order per vendor, decrements stock, and attaches all resulting order identifiers to a single payment document.

Payment. The user selects online payment and completes the gateway's hosted checkout. Technically, the server creates a gateway order for the exact amount and returns its identifier; after the user pays, the client posts back the payment identifier and signature, which the server verifies using an HMAC

computed with its own secret. Only a verified signature marks the payment successful and releases the orders to vendors.

Order acceptance and preparation. The vendor sees a new order appear on the dashboard with an audible alert, accepts it and marks it ready when prepared. Technically, each action issues a PATCH to the order's status endpoint, which validates the transition against the state machine and the actor's role, appends to the embedded status history, and emits a Socket.IO event to the order room.

Delivery. A delivery partner on duty sees the task, accepts it, collects from the vendor and delivers to the block, entering the customer's OTP at handover. Technically, acceptance performs an atomic conditional update so that only one partner can claim a task; each subsequent status change is timestamped; and the delivery is completed only when the submitted OTP matches the stored value, after which the partner's earnings ledger is credited.

Resale listings. A student lists a used calculator with photographs, a condition grade and an asking price; an administrator approves it, after which it appears in the catalogue. Technically, the listing is a product owned by an automatically created student-seller vendor profile, flagged as resale so that it is subject to approval and excluded from commercial analytics.

Reviews. After delivery, the customer is prompted to rate the vendor, the partner and the items. Technically, a uniqueness constraint on order and target prevents duplicate reviews, and an aggregation pipeline recomputes the affected average ratings.

Analytics. A vendor opens the dashboard and sees revenue by day, orders by hour and the top ten items. Technically, aggregation pipelines group the vendor's orders by the relevant key, with results cached briefly to avoid recomputation on every dashboard load.

CHAPTER – 8 TESTING

Testing was performed continuously rather than as a final phase. Each module was tested in isolation as it was built, integrated flows were tested at the end of each development sprint, and a complete user-acceptance round was conducted with real students and a campus vendor before submission.

8.1 Postman Testing

A Postman collection was maintained alongside the API, organised into folders mirroring the resource groups — authentication, vendors, products, cart, orders, payments, deliveries, reviews and admin. Environment variables held the base URL and tokens for each role, so the same collection could be executed against the local, staging and production deployments without modification. A post-response script on the login request captured the returned token into the environment automatically, which removed manual token copying between requests.

Each request carried assertions written in the collection's test scripts, checking the status code, the response schema, and key business invariants — for example, that a checkout containing items from two vendors returns exactly two order identifiers, and that the sum of the sub-order totals equals the amount sent to the payment gateway. Negative cases were tested as deliberately as positive ones: expired tokens, a customer token on a vendor-only route, an order quantity exceeding available stock, an invalid status transition, and a forged payment signature. Running the collection took under a minute and served as a regression check before every merge.

8.2 Integration Testing

Integration tests were written with Jest and Supertest against an in-memory MongoDB instance, so each run started from a clean, deterministic state. Rather than testing endpoints in isolation, these tests exercised complete journeys: register and verify a customer, approve a vendor, publish products, build a mixed cart, check out, verify a simulated payment, accept and prepare each sub-order, assign a delivery, complete it with an OTP, and post reviews — asserting the state of the database after each step.

Particular attention was given to concurrency and failure paths. A test issues two simultaneous checkouts for the last remaining unit of a product and asserts that exactly one succeeds and stock never becomes negative. Another test simulates a payment gateway failure mid-transaction and asserts that no order is left in a half-created state. A third test has two delivery partners accept the same task simultaneously and asserts that only one assignment persists.

8.3 Tools Used for Testing

- Postman — manual and scripted API testing with per-environment collections and assertions.
- Jest — unit testing of services, validators and utility functions.
- Supertest — HTTP-level integration testing of Express routes.
- mongodb-memory-server — isolated, disposable database instance per test run.
- React Testing Library — component behaviour tests written from the user's perspective.
- Chrome DevTools and Lighthouse — performance, accessibility and responsive-layout auditing.

- GitHub Actions — automated execution of the full suite on every push and pull request.
- Manual user-acceptance testing — a structured session with twenty students and one campus vendor.

8.4 Test Cases and Results

ID	Test Case	Expected Result	Status
TC-01	Register with a non-institutional email address	Rejected with a validation error	Pass
TC-02	Log in before completing email verification	Blocked with a prompt to verify	Pass
TC-03	Access the vendor dashboard using a customer token	403 Forbidden	Pass
TC-04	Search the catalogue by keyword and filter by price	Correct, paginated, sorted results	Pass
TC-05	Add items from three vendors to one cart	Cart shows three vendor groups with correct subtotals	Pass
TC-06	Check out a mixed cart	Three sub-orders created, one payment record	Pass
TC-07	Order a quantity greater than available stock	Rejected; stock unchanged	Pass
TC-08	Two users check out the last unit simultaneously	One succeeds, one fails; stock never negative	Pass
TC-09	Submit a forged payment signature	Verification fails; orders not released	Pass
TC-10	Vendor marks an order READY	Status updated; delivery task published; customer notified live	Pass
TC-11	Two partners accept the same delivery task	Only the first assignment persists	Pass
TC-12	Complete delivery with an incorrect OTP	Rejected; order remains out for delivery	Pass
TC-13	Customer attempts to mark an order delivered	Transition rejected for that role	Pass
TC-14	Cancel an order before vendor acceptance	Order cancelled; stock restored; refund initiated	Pass
TC-15	Submit a second review for the same order	Rejected by uniqueness constraint	Pass
TC-16	Load the catalogue on a throttled 3G connection	Interactive within acceptable time with skeleton states	Pass
TC-17	Use the application at 360 px viewport width	No horizontal overflow; all actions reachable	Pass
TC-18	Administrator approves a pending vendor	Vendor becomes visible in the catalogue	Pass

All listed cases passed in the final round. Two defects found in earlier rounds — a race condition in stock decrement and a token refresh loop that could fire repeatedly on a failed refresh — were fixed and are covered by TC-08 and by a dedicated unit test respectively. Lighthouse audits of the deployed build returned performance and accessibility scores above ninety on a simulated mobile connection.

CHAPTER – 9 DEPLOYMENT

9.1 Steps to Deploy

1. Provision a MongoDB Atlas cluster, create a database user with least-privilege access, and restrict network access to the API service.
2. Create a Cloudinary account and obtain the cloud name, API key and API secret for image storage.
3. Obtain payment gateway test credentials and register the callback URL of the deployed API.
4. Push the reviewed code to the main branch of the GitHub repository.
5. Create a web service on the hosting platform pointing at the server directory, with the build command installing dependencies and the start command launching the Express application.
6. Configure all server environment variables on the platform, including the database URI, JWT secrets, gateway keys, Cloudinary credentials and SMTP settings.
7. Deploy the API and confirm that the health-check endpoint responds successfully.
8. Create a static site for the client directory, with the build command producing the production bundle and the publish directory set to the build output.
9. Set the client environment variable holding the deployed API base URL and rebuild.
10. Update the server's CORS allow-list with the deployed client origin.
11. Run the Postman collection against the production environment as a smoke test.
12. Seed reference data — categories, campus blocks and an initial administrator account.
13. Enable automatic deployment on merges to main and confirm that the CI pipeline gates on tests.

9.2 Environment Configuration

Configuration is supplied entirely through environment variables and never committed to version control. An `.env.example` file documents every required key so that a new developer can reproduce the environment without receiving secrets. Three environments are maintained — development, staging and production — sharing one codebase and differing only in configuration.

Variable	Purpose
PORT	Port on which the Express server listens
NODE_ENV	Environment name controlling logging verbosity and error detail
MONGO_URI	Connection string for the MongoDB cluster
JWT_ACCESS_SECRET / JWT_REFRESH_SECRET	Signing secrets for access and refresh tokens
ACCESS_TOKEN_TTL / REFRESH_TOKEN_TTL	Token lifetimes
CLOUDINARY_CLOUD_NAME / _API_KEY / _API_SECRET	Image storage credentials
RAZORPAY_KEY_ID / RAZORPAY_KEY_SECRET	Payment gateway credentials
SMTP_HOST / SMTP_USER / SMTP_PASS	Transactional email credentials
CLIENT_ORIGIN	Allowed origin for CORS
ALLOWED_EMAIL_DOMAINS	Institutional domains accepted at registration

VITE_API_BASE_URL	API base URL compiled into the client bundle
-------------------	--

9.3 Hosting of Front-End and Back-End

The front end is deployed as a static bundle to a CDN-backed hosting platform, which provides automatic TLS certificates, immutable atomic deployments, instant rollback and global edge caching. Because the bundle contains no secrets, it can be cached aggressively; only the API base URL is compiled into it.

The back end runs as a containerised web service on a managed platform that supervises the process, performs health checks on a dedicated endpoint, restarts on failure and can scale horizontally behind a load balancer. Because the API is stateless — sessions live in signed tokens rather than server memory — additional instances can be added without sticky sessions or shared session storage. The WebSocket server runs in the same process, with a Redis adapter identified as the mechanism for multi-instance event propagation should scale require it.

The database is hosted on MongoDB Atlas with daily automated backups, point-in-time recovery, IP allow-listing and query performance monitoring. Deployment health is observed through the platform's logging and metrics dashboards, with alerts configured on elevated error rates and on response times exceeding the agreed threshold.

CHAPTER – 10 CHALLENGES AND LIMITATIONS

10.1 Issues Faced During Development

Splitting a single checkout across multiple vendors proved to be the most conceptually difficult part of the project. An early design modelled one order per checkout with a vendor field on each line item, which made status tracking incoherent: there was no valid single status for an order whose stationery items had been collected but whose food was still being cooked.

Stock consistency under concurrency was the second significant issue. During load testing, two simultaneous checkouts for the last unit of a popular item both succeeded, driving stock negative, because the read of the stock level and the write of the decrement were separate operations.

Payment verification presented a security challenge. An initial implementation marked an order paid on the basis of a success callback from the browser, which a user could trivially forge with a crafted request.

Real-time updates were initially implemented by polling the order endpoint every few seconds. With a few hundred concurrent users during a lunch rush this produced a large volume of near-identical requests and noticeably degraded API response times.

Performance on the campus network was worse than on the development machines. Product images uploaded directly by vendors were often several megabytes, and the initial JavaScript bundle included all four dashboards, making the first load slow on congested Wi-Fi.

Non-technical difficulties were also real. Campus vendors were initially reluctant to adopt a digital dashboard, and the first version of the vendor interface — designed by developers for developers — was found in testing to be too dense for a shopkeeper managing a lunch rush.

10.2 Solutions Applied

The order model was restructured so that a checkout produces one order document per vendor, all referencing a single payment document. Each sub-order then carries its own independent status, while the customer still experiences one cart and one payment. This change removed an entire class of state ambiguity.

Stock consistency was resolved by moving the read-modify-write into an atomic conditional update that decrements stock only when sufficient quantity is present, executed inside a database transaction together with order creation. If any item in the checkout fails the condition, the entire transaction aborts and no partial order is created.

Payment verification was moved entirely to the server. The gateway order is created server-side for an amount computed from the database, and the returned payment identifier and signature are verified with an HMAC computed using the server's secret. A client-reported success is now treated as a claim, not as evidence.

Polling was replaced with Socket.IO. Clients subscribe to rooms for the orders that concern them, and the server emits events only on actual state changes. Request volume during peak testing fell substantially and updates became perceptibly faster.

Performance was addressed on several fronts: images are uploaded to Cloudinary and served in compressed, width-appropriate formats through its CDN; route bundles are code-split so that a customer downloads only the customer interface; list components are memoised and paginated; and skeleton loaders were introduced so that the interface feels responsive even while data is in flight.

The vendor interface was redesigned after the first round of vendor testing into a single-screen queue with large controls and only three primary actions — accept, mark ready, mark handed over. Two short training sessions were conducted with participating vendors, and a printed one-page guide was provided.

10.3 Current Limitations and Known Issues

- Payment integration operates in sandbox mode; live settlement requires a merchant account held by the institution, together with a vendor payout and reconciliation workflow that is not yet implemented.
- Delivery tracking reports discrete status milestones rather than continuous GPS position, since indoor campus positioning is unreliable and continuous location sharing raises privacy concerns.
- Delivery partner assignment is pull-based — the first partner to accept a task wins — rather than optimally allocated by proximity and current load.
- There are no native mobile applications; the deliverable is a responsive web application, which cannot yet deliver true push notifications on all platforms.
- Recommendations are limited to popularity and category rules; no collaborative filtering or machine-learning personalisation is implemented.
- The interface is available in English only.
- Analytics are computed on demand from live aggregation pipelines with short-lived caching, which would require a pre-aggregated reporting store at substantially higher data volumes.
- Dispute resolution is manual, requiring an administrator to review and decide each case.
- The system has been tested with a controlled group of users and a small number of vendors; behaviour at full campus scale during a peak lunch rush has been simulated through load testing but not yet observed in production.

CHAPTER – 11 FUTURE ENHANCEMENTS

11.1 Planned Features

- Native Android and iOS applications built with React Native, sharing business logic with the existing web client and enabling reliable push notifications.
- Live delivery tracking on a campus map, using periodic position updates shared only for the duration of an active delivery and with explicit partner consent.
- Intelligent delivery allocation that assigns tasks by proximity, current load and partner rating, and batches multiple orders destined for the same block into one trip.
- Scheduled and recurring orders, allowing a student to pre-book breakfast for an entire week or a laundry pickup every Friday.
- Group ordering, in which several students in a hostel room contribute to a single cart with automatic bill splitting.
- A wallet and subscription model offering a monthly delivery pass and prepaid balance with faster checkout.
- Vendor payout and reconciliation automation, with commission deduction, settlement scheduling and downloadable statements.
- An in-app chat between buyer and seller for resale negotiation, with moderation and message retention policies.
- Multilingual support, beginning with the regional language of the institution.
- Accessibility certification against WCAG 2.1 AA, including full screen-reader support and keyboard navigation.

11.2 Possible Integrations and Optimisations

Institutional integration is the most valuable direction. Single sign-on through the university identity provider would remove separate registration entirely and guarantee that every account maps to a current member of the institution. Integration with the student identity card, whether through QR code or NFC, would allow contactless pickup verification. Integration with the timetable system would allow the platform to propose delivery windows that fall inside a student's actual free periods.

Machine learning offers several practical applications: demand forecasting that tells a canteen how many plates of each item to prepare for a given day, personalised recommendations based on order history, dynamic estimation of preparation and delivery time from observed data rather than fixed values, and anomaly detection for fraudulent orders or review manipulation.

On the engineering side, several optimisations are identified. A Redis layer would cache the catalogue and session data and would act as the Socket.IO adapter required for multi-instance scaling. A pre-aggregated reporting collection, updated by a scheduled job, would make analytics constant-time regardless of order volume. Converting the client into a full progressive web application with a service worker would allow offline catalogue browsing and installability. Containerising all services with Docker and orchestrating them would simplify horizontal scaling. Finally, an event-driven architecture

in which order events are published to a queue would decouple notifications, analytics and payout processing from the request path.

A longer-term possibility is a multi-campus deployment, in which the platform operates as a tenanted system serving several institutions from one codebase, with each campus retaining its own vendors, users, catalogue and administrators.

CHAPTER – 12 CONCLUSION

12.1 Summary of the Project

This project set out to address a problem that is visible on almost every university campus: a dense and continuous volume of commerce conducted without any coordinating digital system. Students queue for food during short breaks, vendors accept orders verbally and keep no records, second-hand equipment is discarded rather than resold, and the institution has no visibility into any of it. External commercial platforms do not solve this problem, because they are modelled on cities rather than campuses.

The Multi-Vendor Campus Marketplace & Delivery Platform was designed and developed as a direct response. It is a web-based system on which multiple independent vendors operate their own storefronts under a single unified catalogue, customers order across vendors in one transaction, student delivery partners fulfil orders within the campus, and administrators oversee the whole. It was built with React.js, Node.js, Express.js and MongoDB, secured with token-based authentication and role-based authorisation, extended with a payment gateway and real-time communication, tested at unit, integration and acceptance levels, and deployed on managed cloud infrastructure with continuous integration.

12.2 What Was Achieved

- A fully functional, deployed multi-vendor marketplace with four distinct, role-secured user interfaces.
- A working multi-vendor cart that splits one customer checkout into independent vendor-wise orders backed by a single payment.
- A transactionally safe order and stock engine that remains consistent under concurrent load.
- A complete delivery workflow with task publication, atomic assignment, real-time status and OTP-verified handover.
- Verified online payment with server-side signature validation, plus a cash-on-delivery path.
- A peer-to-peer resale sub-module built on the same catalogue and trust infrastructure.
- Analytics dashboards providing vendors and administrators with actionable demand and performance data.
- A documented REST API, a maintained Postman collection and an automated test suite executed on every push.
- Validation with real users: a structured acceptance session with twenty students and a participating campus vendor confirmed that the core journeys are usable without instruction.

12.3 Skills Learned During Development

Technically, the team gained practical, end-to-end experience of full-stack development: component architecture and state management in React, REST API design and layered server architecture in Node.js and Express, schema design and aggregation in MongoDB, and the integration of third-party payment, storage, email and real-time services. Working on the security of a system that handles money

taught lessons that no tutorial conveys as forcefully — above all, that the client can never be trusted, and that every price, total and permission must be re-derived on the server.

The project also developed engineering discipline that goes beyond writing code. Collaborating through branches and pull requests on a shared repository required communication and code review. Writing tests for concurrency failures required reasoning about states that do not occur in ordinary manual testing. Deploying to the cloud exposed the gap between code that works on a development machine and code that works under real network and configuration constraints.

Finally, the project taught a lesson in user-centred design. The first vendor dashboard was rejected in testing not because it lacked functionality but because it demanded too much attention from a person who had thirty seconds to spare. Rebuilding it around the vendor's actual working conditions, rather than around the data model, was the single change that most improved the system. Requirements gathering, iteration on feedback, time and task planning across a three-member team, and technical documentation were all exercised in the course of the work, and are among the most transferable outcomes of the project.

CHAPTER – 13 REFERENCES

1. React Documentation — Meta Open Source. <https://react.dev>
2. Node.js Official Documentation. <https://nodejs.org/en/docs>
3. Express.js Guide and API Reference. <https://expressjs.com>
4. MongoDB Manual and Aggregation Documentation. <https://www.mongodb.com/docs>
5. Mongoose ODM Documentation. <https://mongoosejs.com/docs>
6. MDN Web Docs — HTML, CSS, JavaScript and Web APIs, Mozilla. <https://developer.mozilla.org>
7. Tailwind CSS Documentation. <https://tailwindcss.com/docs>
8. JSON Web Token Introduction and Best Practices. <https://jwt.io/introduction>
9. OWASP Top Ten Web Application Security Risks. <https://owasp.org/www-project-top-ten>
10. Razorpay Payment Gateway Developer Documentation. <https://razorpay.com/docs>
11. Socket.IO Documentation. <https://socket.io/docs>
12. Cloudinary Developer Documentation. <https://cloudinary.com/documentation>
13. Jest Testing Framework Documentation. <https://jestjs.io/docs>
14. Postman Learning Center. <https://learning.postman.com>
15. Git Documentation and Pro Git, Chacon S. and Straub B., Apress. <https://git-scm.com/doc>
16. Elmasri R. and Navathe S. B., Fundamentals of Database Systems, 7th Edition, Pearson.
17. Pressman R. S. and Maxim B. R., Software Engineering: A Practitioner's Approach, 8th Edition, McGraw-Hill.
18. Sommerville I., Software Engineering, 10th Edition, Pearson.
19. Nielsen J., Usability Engineering, Morgan Kaufmann.
20. MongoDB Atlas and Render Deployment Guides. <https://www.mongodb.com/docs/atlas>, <https://render.com/docs>

CHAPTER – 14 APPENDICES

Appendix A — Screenshots of the Application

The following screens are to be inserted here from the deployed application. Each screenshot should be captioned and numbered.

- A.1 Landing page and login screen
- A.2 Registration with institutional email verification
- A.3 Unified multi-vendor catalogue with filters applied
- A.4 Product detail page with vendor information and reviews
- A.5 Multi-vendor cart showing vendor-wise grouping
- A.6 Checkout and payment gateway screen
- A.7 Customer order tracking timeline
- A.8 Vendor dashboard order queue
- A.9 Vendor product management and stock control
- A.10 Vendor analytics dashboard
- A.11 Delivery partner task pool and active delivery screen
- A.12 OTP verification at handover
- A.13 Administrator approval console
- A.14 Resale listing page
- A.15 Mobile responsive views

Appendix B — Sample Code Snippets

B.1 Authentication middleware verifying the access token and attaching the user to the request.

B.2 Role guard middleware restricting a route to a specified set of roles.

B.3 Order service function that splits a multi-vendor cart into vendor-wise orders inside a database transaction, with atomic conditional stock decrement.

B.4 Payment signature verification using an HMAC computed with the gateway secret.

B.5 Order status transition validator implementing the permitted-transitions map.

B.6 Mongoose product schema including the text index and flexible attributes object.

B.7 Aggregation pipeline producing vendor revenue grouped by day and by hour.

B.8 Socket.IO room subscription and status broadcast on the server, with the corresponding client listener.

B.9 Axios instance with request and response interceptors handling token attachment and silent refresh.

Note: the complete source code is maintained in the project's GitHub repository; the snippets above are reproduced in the printed appendix with line numbering and explanatory comments.

Appendix C — Installation and Setup Instructions

1. Install Node.js v18 LTS or higher and Git on the development machine.
2. Clone the repository and change into the project directory.
3. Install server dependencies from within the server directory using `npm install`.
4. Copy `.env.example` to `.env` in the server directory and populate every value listed in Section 9.2.
5. Ensure MongoDB is running locally, or supply an Atlas connection string in `MONGO_URI`.
6. Start the API in development mode with `npm run dev` and confirm the health endpoint responds.
7. Install client dependencies from within the client directory using `npm install`.
8. Create the client `.env` file containing `VITE_API_BASE_URL` pointing at the running API.
9. Start the client development server with `npm run dev` and open the printed local URL.
10. Run the seed script to create categories, campus blocks and an administrator account.
11. Log in as the administrator, approve a vendor account, and add sample products to verify the installation.

Appendix D — User Manual

For customers. Register with the institutional email address and complete verification through the emailed link. Browse or search the catalogue, use filters to narrow results, and compare prices across vendors. Add items from any number of vendors to the cart. At checkout, select the delivery block and room, choose online payment or cash on delivery, and confirm. Track the order on the tracking screen, which updates automatically. On arrival, share the displayed OTP with the delivery partner. After delivery, rate the vendor, the partner and the items.

For vendors. Register as a vendor and wait for administrator approval. Once approved, complete the storefront profile with shop name, category, location and operating hours. Add products with photographs, prices and stock quantities. Watch the order queue during operating hours: accept incoming orders, mark them as preparing, and mark them ready when complete. Review the analytics dashboard to plan stock. Toggle the storefront closed when not operating.

For delivery partners. Register as a delivery partner and wait for approval. Go on duty to see available tasks with pickup vendor, drop block and payout. Accept a task, collect the order from the vendor, mark it picked up, deliver it to the stated location and enter the customer's OTP to complete. Earnings are visible in the partner ledger.

For administrators. Review and approve pending vendor, delivery partner and resale listings. Manage users and handle suspensions where required. Review disputes raised against orders and record the resolution. Monitor platform analytics covering order volume, revenue, vendor performance and delivery times.

Appendix E — Geo-Tagged Photographs and Review Forms

E.1 Geo-tagged photographs of the team with the project guide during review sessions are to be attached here.

E.2 Photographs taken during field study and user-acceptance testing with campus vendors and students.

E.3 Review form for Review-1, with the guide's remarks and signature.

E.4 Review form for Review-2, with the guide's remarks and signature.

E.5 Final review form, with the guide's and the course coordinator's remarks and signatures.

Signature of the Guide

(Mrs. Divya Priya Degala)